Motion Planning

Arihant Jain
Electrical and Computer Engineering
UC San Diego

***Abstract*—This project covers the design and implementation of algorithms for motion planning in static and dynamic spaces, and for collision checking with axis-aligned bounding boxes (AABB). We attempt to design a planner that avoids collisions and can successfully plan in complex 3-D environments, which a baseline greedy planner cannot achieve. Our method implements the Rapidly Exploring Random Tree (RRT) algorithm, taking advantage of existing architectures.**

I. Introduction

The goal of this paper is to explore methods of solving planning problems in complex 3-D environments which can be both static and dynamic, under differing time constraints. We first implement a lightweight AABB collision detection algorithm to test valid states and paths, and utilize this algorithm in our RRT implementation to ensure valid sampling. We will then use a traditional sampling-based algorithm, the Rapidly Exploring Random Tree (RRT) algorithm, along with some modifications to save the tree to improve performance in the dynamic case.

Sampling-based algorithms, although suboptimal for path length, often perform better than their search based counterparts, such as A^* or its variants. This is not only due to the improved resolution afforded by sampling based algorithms, but also various optimizations that can be made to speed up tree formation and node connection. This ensures that the algorithm will find a path to the desired goal in a given time, along with asymptotic guarantees to finding the optimal path. Our RRT implementation takes advantage of this speed for planning to the dynamic targets, along with further improvements for speedup.

II. Problem Formulation

A. Collision Checking

For state collision checking, we must check if the state considered is within the boundary of the state space, and if it is not inside any AABBs. For path collision checking, we must consider if any point along the line between two states is out of bounds or inside any AABB. The implementation will be discussed in Section III A.

B. Motion Planning

All the static environments are 3-dimensional, with a single start and goal state.

State Space: $X \in \mathbb{R}^3$, which is the robot position in 3D coordinates.

When framing this as a Deterministic Shortest Path (DSP) problem, we have:

Our list of states V, which in this case is the same as our state space X.

Our list of edges E, which contains pairs of states from V which can be connected feasibly.

Path: any sequence of the form $i_{1:q} = \{i_1, i_2, ..., i_q\}$

All paths from start to goal $P_{\{s,g\}} = \{i_{1:q} | i_k \in V, i_1 \in s, i_q \in g\}$

is a group of paths that leads from the start to the goal with no collisions.

The path length is defined as the sum of the euclidean distances between two consecutive nodes in the path:

$$J^{i_{1:q}} = \Sigma_{k=1}^{q-1} c_{i_k, i_{k+1}}$$

The objective is to find a path from x_s to x_g which minimizes distance:

$$\min_{i_{1:q} \in P_{\{s,g\}}} J^{i_{1:q}}$$

For the dynamic environments, we consider multiple goal locations by changing the goal location for each new location after planning to the prior one. We thus take advantage of the existing graph for increased performance.

III. Methods and Approach

A. Collision Checking

For the collision-checking problem, we take inspiration from the Group Marching Tree paper[2] to implement the collision checking. What follows is pseudo-code for the collision-checking algorithm used to check states and paths.

Collision Checking

$s_0 \rightarrow$ first state, $s_1 \rightarrow$ second state
$boundary \rightarrow$ boundary of state spaces
$blocks \rightarrow$ Axis-aligned bounding boxes
if $s_0 < boundary.min$ or $s_0 < boundary.max$ or
$s_1 < boundary.min$ or $s_1 < boundary.max$
  $collision = True$
end if
for each block in blocks
  if $s_0 < block.min$ or $s_0 < block.max$ or
  $s_1 < block.min$ or $s_1 < block.max$
    $collision = True$
  end if
$slope = s_0 - s_1$
for each block in blocks:
  for each face in $block$
    $t = (face_coord - s_0)/slope$
    if $0 \leq t \leq 1$ (point is in line between s_0 and s_1):
      $projection = s_0 + slope * t$
      if $projection$ is on face:

$collision = True$
end if
end if
return $collision$

This collision checker can now be used with our planning algorithm to ensure valid states and paths.

B. Motion Planning

The RRT algorithm I implemented is as follows:

RRT

s: start state
g: goal state
T: max time
t: time elapsed
$V \leftarrow \{s\}$
$parents \leftarrow \{\}$
while g not in V or $t < T$:
  $x_random \leftarrow SampleFree()$
  $x_nearest \leftarrow FindNearest(x_random, V)$
  $x_new \leftarrow Steer(x_nearest, x_random)$
  if $CollisionFree(x_new, x_nearest)$:
    $V \leftarrow x_new$
    $parents[x_new] \leftarrow x_nearest$
  end if
return $V, parents$

FindNearest

$dist$: euclidean distance between a state and x_random
return $\arg\min_{x \in V} dist(x, x_random)$

Steer

$step$: step distance to steer the random point to
$diff = x_nearest - x_random$
$dist$: euclidean distance between the two states
return $nearest - step * diff/dist$

We can then build the graph using the parents dictionary to find the path from the goal point to the start point.

Since the static environments were more complex, I used the RRT algorithm implemented by OMPL for part 2, the static environments. This is due to the improved speed and ability to find solutions for the more complex maze and monza environments. I used my custom State and Motion Validity checkers along with the ompl planner, and initialized the static environment with one start and goal state.

I used my own implementation of RRT to find the solutions for part 3, the dynamic environments. My code for both is the exact same, with the only difference being the implementation of the planner itself. I used my custom State and Motion Validity checkers along with my custom planner, and initialized the static environment with one start and goal state for every dynamic state. I then re-used the same planner with a new goal state to take advantage of the existing tree and graph formed to improve planning speed and efficiency.

IV. Results

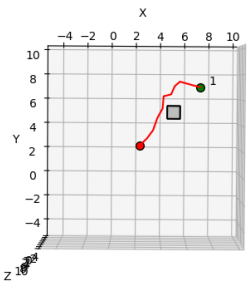
In my discussions with Prof Atanasov, he agreed that using the faster OMPL RRT implementation was valid for the static goal problems, and since my custom RRT implementation found the solution for the dynamic environments, I could use it there and not in the more complex static environments.

Thus, the main drawback of my implementation was the speed being suboptimal due to usage of python and other minor optimization mistakes in my code.

In my custom RRT implementation, I also implemented a goal bias of 5% to ensure that the tree grows in the general direction of the goal during random sampling. Finally, I had to tune my steer distance to achieve maximum efficiency, and I settled on a step of 2m.

A. Static Environments

I have pasted the outputs of each test, to show the planning times and path length, along with images to show the path that was generated.



Running single cube test...

Planning took: 0.0399785041809082 sec.

Success: True

Path length: 10

When using a steer distance of 2 instead of the default 1, we get:

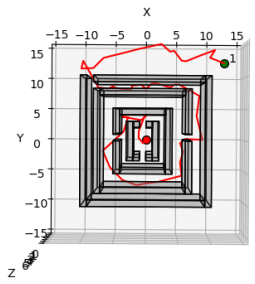
Planning took: 0.014481306076049805 sec.

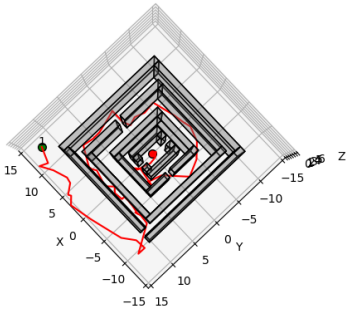
This shows that a higher steer distance is actually beneficial for simpler environments, and should be tuned carefully as such.

When using goal biasing of 50% instead of 5%, we get:

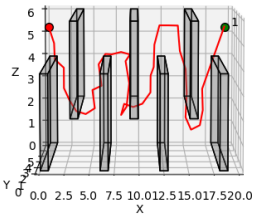
Planning took: 0.0025703907012939453 sec.

Which also makes sense for a simpler environment, since goal biasing will generate successful paths much quicker.





Running maze test...
Planning took: 1.2027180194854736 sec.
Success: True
Path length: 132

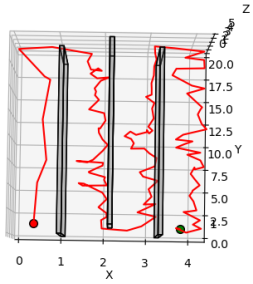


Running flappy bird test...
Planning took: 0.2592945098876953 sec.
Success: True
Path length: 41

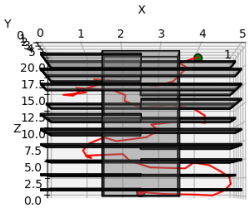
When using a steer distance of 2 instead of 1, we get:
Planning took: 0.5032713413238525 sec.
Path length: 46
This is due to the fact that in slightly more complex environments, a higher steer step results in more failed steers and dilutes path quality.

When using goal biasing of 50% instead of 5%, we get:
Planning took: 1.320420742034912 sec. Path length: 44

We can see that it not only takes longer, but is also a worse path. This is because goal biasing in an environment when the goal is behind some obstacle will always result in a worse path.



Running monza test...
Planning took: 2.2770142555236816 sec.
Success: True
Path length: 106



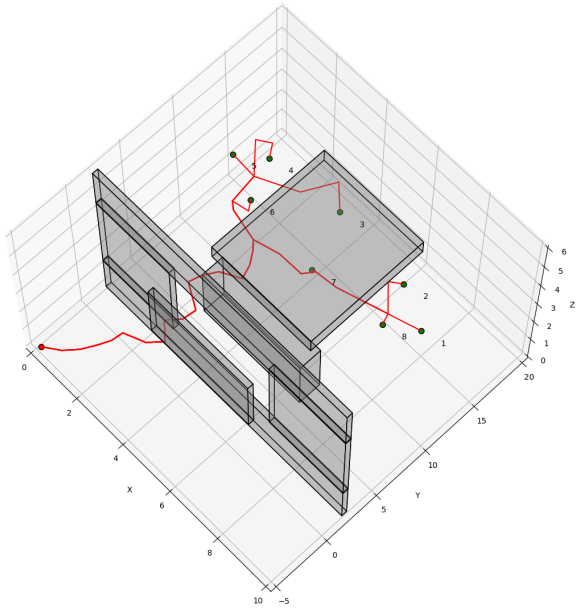
Running tower test...
Planning took: 3.003394603729248 sec.
Success: True
Path length: 44

When using a steer distance of 2 instead of 1, we get:
Planning took: 0.13997840881347656 sec.
Which yet again shows the importance of choosing a good steer distance with RRT when planing.

When using goal biasing of 50% instead of 5%, we get:
Planning took: 2.9975993633270264 sec.
This reiterates the results from the flappy bird environment, showing that more complex environments need careful tuning of the goal biasing and steer distance.

B. Dynamic Environments

Similar to the static environments, I pasted the command line output which contains planning time and whether we successfully reached the goal, and also the path costs for each goal. We can see the path costs increase for later states since the planner makes use of existing paths going through other states.



Running window test...

$t_1 = 0.013973236083984375$ sec.

$t_2 = 0.012309789657592773$ sec.

$t_3 = 0.003143310546875$ sec.

$t_4 = 0.01775836944580078$ sec.

$t_5 = 0.006730318069458008$ sec.

$t_6 = 0.006434917449951172$ sec.

$t_7 = 0.005365848541259766$ sec.

$t_8 = 0.009675025939941406$ sec.

Success: True

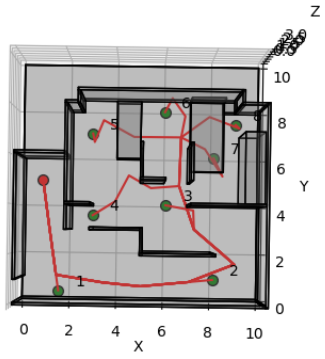
Path length: $[24.24342226584416, 26.444072974894475, 26.144481128266047, 24.91398416630624, 21.87784779436525, 20.314170329043588, 20.676531088404747, 23.618231347545745]$

With a steer distance of 2.5m

Path length: $[24.783254344423874, 25.209265730437725, 28.950272634140074, 25.765815706195735, 26.843059291938317, 22.36152007722983, 22.71769186474731, 22.838764282630315]$

We can see that higher steer distances cause worse planning almost across the board in this more complex dynamic example.

For the room test, I ran into the most trouble with finding a valid path to goal state 4 in time, which is refleced by the planning time being much higher than all the other states. This is due to the fact that state 4 is significantly further away in the z axis as compared to the rest of the points.



Running room test...

$t_1 = 0.0025904178619384766$ sec.

$t_2 = 0.011378765106201172$ sec.

$t_3 = 0.01394796371459961$ sec.

$t_4 = 0.0756981372833252$ sec.

$t_5 = 0.0009729862213134766$ sec.

$t_6 = 0.013785362243652344$ sec.

$t_7 = 0.027016401290893555$ sec.

$t_8 = 0.008901119232177734$ sec.

Success: True

Path length: $[4.648939323595304, 10.594136958915602, 16.988188905280257, 21.71325893323059, 24.75270376425163, 21.849631810367427, 22.611301881616825, 22.132862269882075]$

With a steer distance of 2.5m, i get the following results:

Path length: $[4.558508528016592, 11.848880756870251, 17.15320468136589, 6.116010137400718, 11.954839137711263, 13.720610603533007, 15.930121377283337, 18.653660334088862]$

We can see that a higher steer distance actually results in worse planning for the more complex goal points, but is better in simple path finding when the goal biasing takes place.

V. Conclusion

In this report, we implemented a collision-checking method for AABBs, along with the sampling-based planning algorithm RRT to find a path from the start point to goal point in a 3-D environment. We leveraged the existing tree built from the first run of RRT in the dynamic case to reach our planning efficiency goals, and optimized our implementation using the OMPL library to achieve low planning times as shown in Section IV.

We managed to discuss the importance of goal biasing and steer distance with experiments showing its effect on planning performance, both in terms of time and path cost. We successfully illustrated results for both a static and dynamic environment. Future work will involve further optimizations to find paths which are shorter, or higher dimensional environments, and to consider collision-checking for non-aligned meshes in our environment.

REFERENCES

1) Used Prof Atanasov's class notes and links provided in the project description.
2) Ichter, Brian, et al. "Group Marching Tree: Sampling-Based Approximately Optimal Motion Planning on GPUs." ArXiv.org, 2017, arxiv.org/abs/1705.02403.
3) Ioan A. Şucan, Mark Moll, Lydia E. Kavraki, The Open Motion Planning Library, IEEE Robotics & Automation Magazine, 19(4):72–82, December 2012. https://ompl.kavrakilab.org
4) Weihang Guo, Theodoros Tyrovouzis, and Lydia E. Kavraki, Python Bindings for a Large C++ Robotics Library: The Case of OMPL, 2026 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2026.